

PhotonCamera HDR v6

Reverse Engineering Analysis

Full pipeline analysis: HDR merging, denoising (chroma + luma), demosaicing, tone mapping, and edge-aware adaptive filtering

Package	com.particlesdevs.photoncamera
Version	0.97 (versionCode 26112)
compileSdkVersion	36 (Android 16)
minSdkVersion	26 (Android 8.0)
targetSdkVersion	35
Architecture	arm64-v8a, armeabi-v7a
Language	Kotlin/Java (R8 obfuscated) + GLSL shaders
GPU Pipeline	OpenGL ES 3.0+ compute + fragment shaders
Date	2026-07-25

Library	Purpose
liballocator.so	Memory allocation for RAW buffers
libarchive-jni.so	Archive handling
libcamera2native.so	Camera2 API native bridge
libdngCreator.so	DNG RAW file creation
libflacRecorder.so	FLAC audio recording

Table of Contents

1. HDR Pipeline Architecture
2. Frame Merging (PyramidMerging)
3. Denoise Algorithms — Deep Dive
4. HDR Tone Mapping (ExposureFusionBayer2)
5. Demosaicing
6. Auto White Balance (AWB)
7. Noise Model
8. GLSL Shader Inventory
9. Sensor Noise Models (User's Device)
10. Working Modifications Applied

1. HDR Pipeline Architecture

PhotonCamera captures multiple RAW frames and processes them through a GPU compute pipeline. The pipeline operates in two major phases: RAW merging and post-processing.

1.1 Capture Phase (HdrxProcessor)

- **Frame Collection:** Captures multiple RAW frames (typically 15)
- **ISO/Exposure Selection:** IsoExpoSelector pairs each frame with exposure metadata
- **Gyro Stabilization:** Each frame gets a GyroBurst for motion compensation
- **Exposure Classification:** Frames classified as SHORT (underexposed) or LONG (overexposed)
- **Lucky Frame Selection:** Frames sorted by 'unluckiness' metric; worst 25% discarded if >10 frames
- **Reference Frame:** Frame with lowest exposure multiplier chosen as base

1.2 Post-Processing Pipeline (PostPipeline)

After RAW merging, the post-processing pipeline applies the following nodes in order:

#	Node	Domain	Purpose
1	ABLC	RAW	Black Level Correction
2	Bayer2Float	RAW	Convert Bayer uint16 to float
3	Initial	RGB	Color correction, gamma, tonemapping, saturation
4	BayerMoire	Bayer	Moiré pattern removal (SNN on color ratios)
5	ExposureFusionBayer2	Bayer	LTM — Local Tone Mapping (HDR fusion)
6	Demosaic3	Bayer→RGB	Edge-directed Bayer demosaicing
7	AWB	RGB	Auto White Balance correction
8	ESD3D2	RGB	Edge-Preserving Subsampling Denoise 3D
9	BilateralSeparable	RGB	Separable bilateral filter
10	SmartNR	RGB	Smart noise reduction (guided bilateral + chroma median)
11	ChromaticFlow	RGB	Chromatic aberration correction
12	Equalization	RGB	Histogram equalization / adaptive tone curve
13	ColorD	RGB	Color grading
14	Sharpen/Sharpen2	RGB	Sharpening
15	Wavelet	RGB	Wavelet-based processing
16	RotateWatermark	RGB	Rotation + watermark

2. Frame Merging (PyramidMerging)

The core HDR alignment and merge engine. Operates on RAW Bayer data using GPU compute shaders.

2.1 Hot Pixel Detection Pipeline

- **Averaging:** All frames averaged using compute shader (merge/avermix.glsl) — running average: $\text{mix}(\text{current}, \text{new}, 1/\text{frameCount})$
- **Detection (merge/hotpixeldetect.glsl):** 3×3 median filter on averaged Bayer data. Noise model: $\text{noise} = \sqrt{\text{noiseS} \times \text{pixel} + \text{noiseO}}$. Dynamic threshold with adaptive increase as list fills.
- **Statistical Filtering:** If >50 detections, compute mean/stddev of strengths, remove outliers below mean – 1.5× stddev. If >2000 detections, sort by strength and keep top N.
- **Correction (merge/hotpixelcorrect.glsl):** Replace hot pixels with median of neighbors.

2.2 Pyramid Alignment + Merging

- Block-based motion estimation using alignment/align.glsl with 16×16 tiles
- Cosine window weighting for smooth blending across tile boundaries
- Noise-aware weighting: $\text{noise} = \max(\sqrt{\text{bayer} \times \text{noiseS} + \text{noiseO}}, \text{minLevel})$
- Wiener-like shrinkage on frame differences: $\text{diff} \times= \text{noise}^2 / (\text{noise}^2 + |\text{diff}|^2)$
- Overexposure protection: clamps blending when pixels exceed target threshold

3. Denoise Algorithms — Deep Dive

PhotonCamera uses ESD3D2 (Edge-Preserving Subsampling Denoise 3D) as its primary denoise algorithm for both luma and chroma channels. The developer confirmed that only `denoise/esd3d2.glsl` and `denoise/guidedupsample.glsl` are used for actual luma + chroma denoising.

3.1 ESD3D2 — Main Denoise

When 'Use Color Denoising' is enabled, the pipeline runs a low-resolution ESD3D2 pass, guided upsampling, then a full-resolution ESD3D2 pass.

Parameter	Default	Range	Description
enable	true	0/1	Enable ESD3D denoising
luma	0.8	0-2	Luma strength multiplier
moire	1.5	0-5	Moire reduction strength
shadowBoost	0.5	0-2	Shadow denoise boost
noiseTarget	0.00390625	0-0.1	Target noise level for kernel mapping
noiseToKernelSize	24	0-50	Noise to kernel size scaling
minSize	7	1-21	Minimum kernel size
maxSize	9	1-51	Maximum kernel size (changed from 21)
useColorDenoising	true	0/1	Enable chroma denoise path
chromaStrength	1.0	0-1	Chroma denoise threshold multiplier

3.1.1 SNN Filter Algorithm

The ESD3D2 shader uses Hybrid SNN (Symmetric Nearest Neighbor) filtering:

- For each kernel position (i,j), examines 4 symmetric quadrants: (i,j), (-i,-j), (i,-j), (-i,j)
- Computes RGB distance from center pixel to each quadrant: $d = \text{length}(|\text{neighbor} - \text{center}|)$
- Weight formula: $w = 1 - d^2/(d^2 + \text{sig}^2)$ — edge-preserving (high weight for similar pixels)
- Minimum subtraction: $w_m = \min(w[0], w[1], w[2], w[3])$; $w -= w_m$; $w = \text{normalize}(w)$
- Two separate thresholds: sigY for luma filtering, sigZ for chroma filtering
- $\text{sigY} = \text{noiseS} \times \text{factor} + \text{noiseS}^2 \times 3/8 + \text{noiseO}$ (noise-dependent)
- $\text{sigZ} = \max(\text{sigY}, |\text{chromaDiff}| \times \text{moire}) \times \text{chromaStrength}$
- Shadow boost: $\text{sigY} \times= 1 + \text{SHADOWBOOST} \times \text{clamp}(1 - \text{factor} \times 2, 0, 1)$

3.1.2 Edge-Aware Adaptive Kernel (Working Modification)

The SNN filter was modified to detect color boundaries and adaptively shrink the kernel:

- Pre-scan: check 8 perimeter pixels at KSIZE distance from center
- Edge threshold: $3 \times \text{sqrt}(\text{sigY})$, minimum 0.05
- If any perimeter pixel has RGB distance > threshold → color edge detected
- Edge detected → kernel shrinks to KSIZE_SMALL (3) to preserve color
- No edge → kernel stays at full MSIZE for maximum denoising
- This preserves teal text colors while still denoising flat wall surfaces

3.2 Guided Upsample (denoise/guidedupsample.glsl)

Reconstructs full-resolution output from the low-resolution ESD3D2 result using a local linear model. The developer noted this has 'huge space for improvement'.

- Local Linear Model: $\text{output} = a \times \text{guideLuma} + b$, fitted per low-res cell
- Regression uses 3×3 neighborhood with Gaussian weights ($\text{sigma}=1.2$)
- Noise-adaptive regularization: $\text{varThreshold} = 0.1 \times (\text{noiseS} \times \text{avgLuma} + \text{noiseO})$
- Bilinear interpolation of a,b coefficients across 4 nearest low-res cells
- Shadow protection: $\text{smoothstep}(0, 0.001, \text{guideBr})$ in dark areas
- Brightness preservation: $\text{Output} = (\text{Output}/\text{br}) \times \text{guideBr}$

Key Finding: The 5×5 regression window caused color desaturation on small features (like text). The 5×5 window at SCALE=4 covers 20×20 full-res pixels, which spans both teal text and white background even at the text center. Reverting to 3×3 fixed this.

4. HDR Tone Mapping (ExposureFusionBayer2)

Multi-scale Laplacian pyramid fusion for local tone mapping. Operates on Bayer data before demosaicing.

4.1 Exposure Analysis

- Histogram analysis from 4× downsampled image
- Overexposure detection: weighted average of bright pixels (128-240 range)
- Over-expose gain: $\max(128 / (\text{avgBright} + 1), 1.0)$, clamped by noise
- Underexposure detection: gamma-corrected search with gammaKSearch coefficient
- Under-expose gain: $\min(179.2 / (\text{avgDark} + 1), 1.0)$

4.2 Multi-Scale Fusion

- Laplacian pyramid construction with configurable downScalePerLevel (default 2.0)
- Per-level blending weights: well-exposedness (Gaussian PDF) × contrast (Laplacian high-pass)
- Quadratic weight emphasis: $\text{weights} \times = \text{weights}$
- Dehazing: blend factor increases at coarser pyramid levels
- Final fusion map applied to original resolution via ltm/fusionmap.glsl
- Spline-based tone curve with 5 configurable control points, interpolated to 1024-entry LUT

5. Demosaicing

PhotonCamera supports multiple demosaicing algorithms, selectable via the demosaicingMethod tunable.

Method	Description
Demosaic (compatibility)	Basic demosaicing for older devices
Demosaic2	Alternative algorithm
Demosaic3 (default)	Two-pass edge-directed: green first, then R/B guided by green
DemosaicQUAD	For Quad Bayer sensors (4×4 pattern)
MonoDemosaic	Monochrome sensor support
BinnedDemosaic	For binned sensor modes

6. Auto White Balance (AWB)

- Downsample input by 5× for fast analysis
- Extract chroma via awbgetchroma.glsl shader
- Build 16×16×16 RGB histogram
- Optional LUT from awb_lut.png file
- Find peak chromaticity (most frequent color in image)
- Compute correction vector: normalize so peak → neutral white
- Apply via Parameters.P[] with camera2 color correction matrix

7. Noise Model

PhotonCamera uses a Poisson-Gaussian noise model: $\text{noise}(\text{pixel}) = \sqrt{\text{noiseS} \times \text{pixel} + \text{noiseO}}$. The noiseS (shot noise) and noiseO (read noise) values are derived from per-sensor coefficients loaded from sensorspecifics.txt files.

Coefficient	Role	Formula
NoiseModelA	Shot noise (signal-dependent)	$\text{noiseS} = A \times \text{ISO} / (\text{frames} \times 0.9)$
NoiseModelB	Shot noise offset	Contributes to noiseS
NoiseModelC	Read noise (quadratic)	Contributes to noiseO
NoiseModelD	Read noise (offset)	$\text{noiseO} = D / (\text{frames} \times 0.9)$

7.1 Per-Channel Noise Pairs

The NoiseModeler stores per-channel noise pairs: (noiseS, noiseO) for R, G, B channels. These are scaled by frame count during multi-frame merging.

8. GLSL Shader Inventory

The app contains 130+ GLSL shaders in assets/shaders/. Key shaders by category:

Category	Shaders	Purpose
Frame Merging	merge/merge00.glsl, merge/avermix.glsl, merge/hotpixeldetect.glsl, merge/hotpixelcorrect.glsl	Multi-frame alignment and merge
Denoise	denoise/esd3d2.glsl, denoise/guidedupsample.glsl, denoise/bilateralsep.glsl, denoise/bilateral.glsl	Edge-preserving denoising
Demosaic	demosaic/demosaicp1.glsl, demosaic/demosaicp2.glsl, demosaic/demosaicp12.glsl (multiple variants)	Bayer to RGB conversion
Tone Mapping	ltm/exposebayer2.glsl, ltm/fusionbayer3.glsl, ltm/fusionmap.glsl	Local tone mapping fusion
Color	awbgetchroma.glsl, initial.glsl, reinterpretcolors.glsl	White balance and color correction
Sharpening	sharpening/sharpen33.glsl, sharpening/sharpen55.glsl, capturesharpening.glsl	Detail enhancement
Alignment	alignment/align.glsl, alignment/export.glsl, alignment/pack.glsl	Block-based motion estimation

9. Sensor Noise Models (User's Device)

Noise models loaded from sensorspecifics.txt. Adaptive noise models are disabled; using static per-sensor coefficients. Motion mode with 15 frames.

Lens	Sensor	NoiseModelA	NoiseModelD	Notes
1x Main	sensor_2	2.69e-06	3.28e-07	Low noise
0.6x UW	sensor_3	2.69e-06	3.28e-07	Same as main
3x Tele	sensor_4	2.69e-06	3.28e-07	Same as main
4.1x Tele	sensor_5	3.47e-05	3.85e-07	13× higher shot noise; ISO 4000-12800 indoors

9.1 Actual Pipeline Noise Values (from log)

Stage	NoiseS	NoiseO	KernelSize	MSIZE
Pipeline (PostPipeline)	0.01132	1.03e-4	—	—
Low-res ESD3D2 (÷4)	0.00377	8.14e-5	24.8 → capped	9
Full-res ESD3D2	0.01132	2.44e-4	42.3 → capped	9

10. Working Modifications Applied

The following modifications were tested and confirmed working. Each was validated through iterative testing on the user's device.

10.1 ESD3D2: Edge-Aware Adaptive Kernel (esd3d2.glsl)

Problem: The SNN filter with MSIZE=21 (21×21 kernel = 441 pixels) destroyed color at boundaries. Teal text appeared grey because the kernel spanned both teal and white pixels, and the min(w) subtraction + re-normalization caused instability.

Solution: Added edge detection before the SNN loop. Scans 8 perimeter pixels at KSIZE distance. If any pixel has RGB distance $> 3 \times \sqrt{\text{sigY}}$ from center, the kernel shrinks to KSIZE_SMALL (3×3). This preserves color boundaries while still denoising flat areas at full kernel size.

Result: Teal text colors preserved. Flat surfaces (walls) still denoised. Fabric patterns adapt based on local color variance.

Code change in esd3d2.glsl:

```
Added: #define KSIZE_SMALL 3
Added: Pre-scan loop checking perimeter pixels for color edges
Modified: Main SNN loop uses effectiveKSIZE (dynamic) instead of KSIZE (fixed)
```

10.2 ESD3D2: Max Kernel Default Reduced (ESD3D2.smali)

Problem: Default maxSize=21 was too large, causing color destruction even with edge detection.

Solution: Changed default maxSize from 21 to 9 in the @Tunable annotation.

Result: Smaller default kernel reduces cross-boundary averaging. Users can still increase via tunables if needed.

Code change in ESD3D2.smali:

```
Changed: .field maxSize:I defaultValue = 21.0f → defaultValue = 9.0f
```

10.3 ESD3D2: Chroma Strength Tunable (ESD3D2.smali + esd3d2.glsl)

Purpose: Fine-tune chroma denoise intensity independently of luma. Lower values preserve more color; higher values reduce chroma noise.

Implementation: Added chromaStrength @Tunable field (default 1.0, range 0-1, step 0.05). Injected as CHROMASTRENGTH define per-pass: low-res pass uses the tunable value, full-res pass always uses 1.0. Multiplied onto sigZ in the shader.

Code changes:

- ESD3D2.smali: Added .field chromaStrength:F with @Tunable annotation
- ESD3D2.smali: Inject CHROMASTRENGTH before each k() call in d() method
- esd3d2.glsl: Added #define CHROMASTRENGTH 1.0
- esd3d2.glsl: $\text{sigZ} = \max(\text{sigY}, \min(|\text{chromaDiff}| \times \text{MOIRE}, 0.2)) \times \text{CHROMASTRENGTH}$

10.4 Guided Upsample: 3×3 Window + Noise-Adaptive Regularization (guidedupsample.glsl)

Problem: The original 3×3 window with hardcoded varThreshold=0.001 was identified as having 'room for improvement'. The 5×5 window (tested in v2-v8) caused color desaturation on small features like text because

the window spanned color boundaries.

Solution: Kept 3×3 window (small enough to stay within color boundaries) but added noise-adaptive regularization: $\text{varThreshold} = 0.1 \times (\text{noiseS} \times \text{avgLuma} + \text{noiseO})$. This replaces the hardcoded 0.001 with a value that adapts to actual noise levels.

Result: Better noise handling than original (regularization adapts to sensor/ISO) without the color loss from the larger window.

Code change in guidedupsample.glsl:

```
Changed: computeAB() uses 3×3 window (sigma=1.2, original size)
Changed: varThreshold = 0.1 × (noiseS × avgLuma + noiseO) instead of hardcoded 0.001
Preserved: Original outlier rejection (disabled), shadow protection (0.001)
```

10.5 Summary of All Changes

File	Change	Status
esd3d2.glsl	Edge-aware adaptive kernel (pre-scan + dynamic KSIZE)	Working
esd3d2.glsl	CHROMASTRENGTH define × sigZ	Working
esd3d2.glsl	Full RGB distance for SNN weights (original, preserved)	Working
ESD3D2.smali	maxSize default: 21 → 9	Working
ESD3D2.smali	chromaStrength @Tunable field added	Working
ESD3D2.smali	CHROMASTRENGTH injected per-pass (low-res=setting, full-res=1.0)	Working
ESD3D2.smali	.locals 10 → 15 (register space for injections)	Working
guidedupsample.glsl	3×3 window (original size, preserved)	Working
guidedupsample.glsl	Noise-adaptive regularization ($0.1 \times \text{noiseVar}$)	Working

10.6 Changes That Did NOT Work (Reverted)

Change	Why It Failed
Luma-only SNN weights	Reduced distance between teal and white (similar luma), causing MORE color averaging, not less
5×5 guided upsample window	Spanned color boundaries at SCALE=4 (covers 20×20 full-res), causing desaturation on small features like text
Noise-aware kernel weighting ($1/(\text{luma}^2 \times \text{noiseS} + \text{noiseO})$)	Gave dark pixels too much influence, pulling color estimates toward grey
Shadow protection 0.001→0.02	Too wide, caused blurring in shadow areas
Outlier rejection re-enabled	Over-smoothed, caused sharpness loss

Build & Compilation Notes

- Apktool 2.10.0 used for decompilation and recompilation
- Adoptium JDK 17.0.13 for Java runtime
- Android SDK Build-Tools 34.0.0 for zipalign and apksigner
- APK signed with v2/v3 signature scheme (compatible with Android 16)
- Self-signed certificate (not for production distribution)
- All modifications made via smali editing (no Java recompilation needed)
- GLSL shader edits applied directly to assets/shaders/ files

*Document generated from reverse engineering analysis of PhotonCamera HDR v6 APK.
All shader code and pipeline logic extracted from decompiled source.*